HOUSING PRICE PREDICTION

A Project Report

Submitted by:

Adiba Sami 25MDA10050

in partial fulfillment for the award of the degree of

MASTER OF BUSINESS ADMINISTRATION

IN

(DATA SCIENCE AND ARTIFICIAL INTELLIGENCE)

Chandigarh University

Academic Year 2026–2027

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Residential property is one of the largest financial assets most households ever buy or sell. The price of a house or apartment depends on many factors, including its city and locality, built-up area, number of rooms, age, floor, furnishing, parking and nearby infrastructure. These factors do not influence price in a purely linear way, and manual valuations may vary between buyers, sellers and agents.

Machine learning offers a data-driven approach by learning patterns from historical property listings. Regression algorithms can use property attributes to estimate the price of a home that was not seen during training. Artificial Neural Networks (ANNs) are relevant because they can learn non-linear relationships between input features and a numerical target.

This project investigates ANN-based housing price prediction and compares its results with other regression techniques. Analysing property photographs and neighbourhood information is a possible future enhancement and is not part of the current work.

1.2 Problem Statement

Housing prices depend on factors including city, locality category, property type, area, number of bedrooms and bathrooms, age of the building, floor, furnishing, parking and proximity to a metro station. Their relationships with price can be complex. This project addresses the problem of building and evaluating a machine-learning system that estimates housing prices from structured data, and compares an ANN with alternative regression models.

1.3 Project Objectives

- Explore the housing dataset and identify data-quality issues.
- Prepare numerical and categorical features for model training.
- Implement an Artificial Neural Network for regression.
- Evaluate models using MAE, RMSE, and R^2.
- Compare ANN models with Linear Regression and Random Forest.
- Select a model based on test-set performance and save it as a reusable pipeline.
- Identify future improvements, including a web interface and the use of real market data.

1.4 Scope of the Project

The project focuses on structured housing data, data preprocessing, ANN training, model comparison, and test-set evaluation. The ANN was implemented in original-target and log-

target experiments and compared with Linear Regression and Random Forest. The best-performing pipeline is saved so it can be reused for predictions.

The system is an academic prototype. It provides an estimated price based on patterns in the dataset and does not guarantee the price at which a property will be sold. Importantly, the dataset used in this project is a simulated dataset of housing listings generated for the study (Section 2.1); the reported results therefore describe model behaviour on that dataset and should not be read as valuations of the real housing market. A web application and image-based analysis are possible future extensions and have not been implemented.

CHAPTER 2

DATASET AND METHODOLOGY

2.1 Dataset Source and Description

The project uses a simulated dataset of Indian residential property listings. It was generated programmatically with a fixed random seed (42) so that the data can be reproduced exactly. City-wise base prices per square foot, locality tier, property type, furnishing, building age, floor, parking, balconies and proximity to a metro station are combined with random variation to produce a listed price. Missing values were deliberately introduced in several columns to reflect real listing data. The generated file was saved as housing_data.csv.

A simulated dataset was used so that the complete workflow could be built and tested in a controlled setting. The same code can be applied to a real dataset (for example, a public housing-listings dataset) by replacing the data-loading step.

Characteristic	Description
Number of records	2,400
Number of columns	16
Input features	15
Target variable	price (₹)
Task	Regression
Data type	Structured tabular data (simulated)

housing_data.csv ×

```
city,locality_tier,property_type,area_sqft,bedrooms,bathrooms,balconies,age_years,floor,total_floors,
Chennai,Mid,Independent House,1354.0,3,3,3.0,12.0,1.0,3,Unfurnished,1.0,North,0,0,10528000.0,
Bengaluru,Suburban,Villa,3397.0,4,3,1.0,5.0,0.0,2,Semi-Furnished,1.0,North,0,0,27173000.0,
Kolkata,Mid,Independent House,833.0,2,2,2.0,13.0,1.0,3,Semi-Furnished,0.0,West,0,0,4807000.0,
Pune,Suburban,Builder Floor,789.0,2,2,3.0,12.0,0.0,1,Furnished,0.0,East,0,0,3844000.0,
Mumbai,Suburban,Villa,2888.0,5,5,1.0,13.0,0.0,1,Semi-Furnished,1.0,West,1,0,74057000.0,
Ahmedabad,Mid,Apartment,1680.0,4,5,1.0,19.0,12.0,25,Unfurnished,1.0,West,0,1,6979000.0,
Chennai,Mid,Apartment,422.0,2,2,1.0,11.0,1.0,11,Furnished,2.0,West,1,1,3352000.0,
```

Figure 2.1 — Housing dataset (first rows of housing_data.csv)

The columns include city, locality_tier, property_type, area_sqft, bedrooms, bathrooms, balconies, age_years, floor, total_floors, furnishing, parking, facing, near_metro, lift and price.

2.2 Features and Target Variable

The input features cover property identity, size, age and floor, amenities, and location. The target variable is price. The models learn relationships between these attributes and price to estimate values for held-out records.

Feature group	Examples
Location	city, locality_tier, near_metro

Feature group	Examples
Property identity	property_type, facing
Size and layout	area_sqft, bedrooms, bathrooms, balconies
Age and position	age_years, floor, total_floors
Amenities	furnishing, parking, lift

2.3 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was used to inspect the dataset, review the price distribution, and identify missing values and other quality issues. No duplicate records were identified.

EDA observation	Finding
Records and columns	2,400 records; 16 columns
Minimum listed price	₹13.14 lakh
Median price	Approximately ₹92.41 lakh
Mean price	Approximately ₹142.01 lakh
Maximum listed price	₹35.57 crore
Duplicate records	None identified

The mean being substantially higher than the median indicates a right-skewed price distribution influenced by high-value properties (Figure 2.3). Missing values were observed in parking (195), facing (176), floor (127), age_years (119), balconies (101) and furnishing (83).

The code used for this exploration and its output are shown in Figure 2.2.

01_data_exploration.ipynb ×

```python
import pandas as pd

df = pd.read_csv("housing_data.csv")
print("Dataset shape:", df.shape)
print("Duplicate records:", df.duplicated().sum())

print("\nPrice summary (INR):")
print(df["price"].describe())

print("\nMissing values per column:")
print(df.isnull().sum()[df.isnull().sum() > 0])
```

```
Dataset shape: (2400, 16)
Duplicate records: 0

Price summary (INR):
count    2.400000e+03
mean     1.420086e+07
std      1.724380e+07
min      1.314000e+06
25%      5.955000e+06
50%      9.241000e+06
75%      1.598075e+07
max      3.556810e+08
Name: price, dtype: float64

Missing values per column:
balconies     101
age_years     119
floor         127
furnishing     83
parking       195
facing        176
dtype: int64
```

Figure 2.2 — Exploratory Data Analysis (code and output)

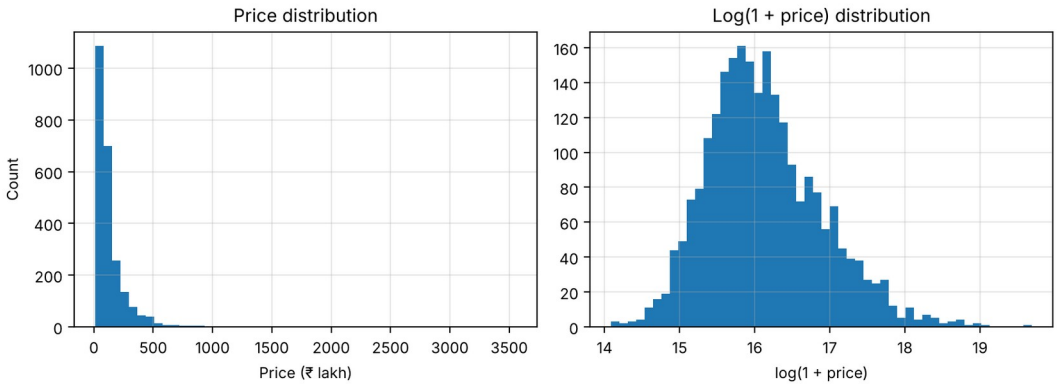


Figure 2.3 — Price distribution and log-transformed price distribution

Median prices differ strongly between cities, with Mumbai highest and Ahmedabad lowest (Figure 2.4). Larger properties generally cost more, but the spread of prices widens with area because city and locality also matter (Figure 2.5).

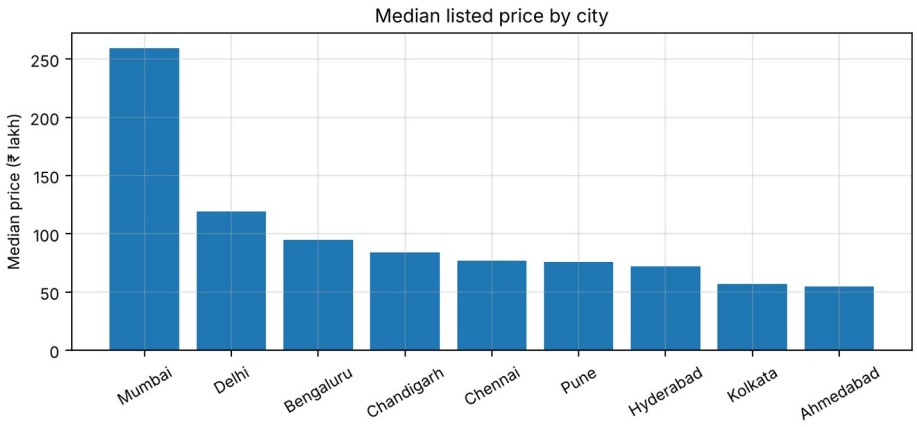


Figure 2.4 — Median listed price by city

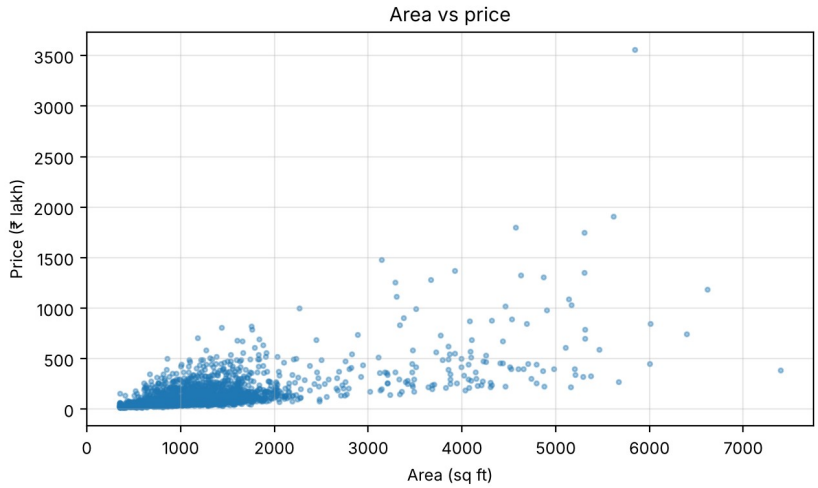


Figure 2.5 — Area versus price

The correlation matrix of numerical features (Figure 2.6) shows that area, bedrooms and bathrooms are strongly related to one another and to price, so the features overlap in the information they carry.

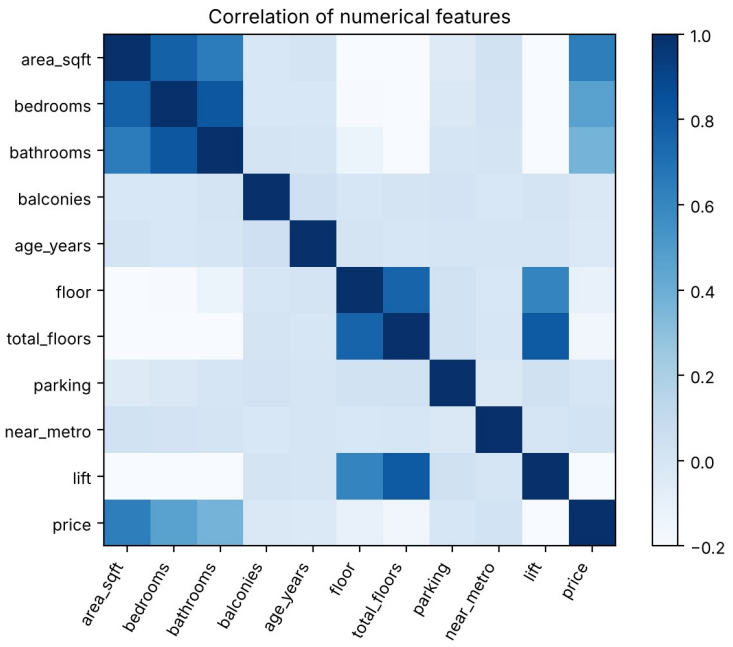


Figure 2.6 — Correlation matrix of numerical features

2.4 Data Preprocessing

Preprocessing was implemented using Pandas and Scikit-learn. The workflow loads the dataset, checks data quality, separates the target variable, identifies numerical and categorical columns, and builds transformation pipelines.

- The generated data was retained separately so preprocessing did not overwrite the source file.
- Missing values were handled inside the machine-learning pipeline rather than filled with arbitrary values before modelling.
- The dataset retained 2,400 records and 16 columns; the target variable had no missing values and no duplicate records were identified.

2.5 Missing Values, Encoding, and Scaling

2.5.1 Numerical Features

The numerical pipeline uses median imputation to replace missing numerical values and StandardScaler to standardise feature scales. Median imputation is less sensitive to extreme observations than mean imputation, while standardisation is useful for the ANN.

2.5.2 Categorical Features

The categorical pipeline uses most-frequent imputation for missing categories and one-hot encoding to represent categories numerically. The encoder uses handle_unknown='ignore' so previously unseen categories do not cause an encoding error during prediction.

A ColumnTransformer combines the numerical and categorical transformations. The preprocessing steps are included with the model pipeline so the same transformations can be applied during training and prediction.

02_data_preprocessing.ipynb ×

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

target = "price"
X, y = df.drop(columns=target), df[target]
num_cols = X.select_dtypes("number").columns.tolist()
cat_cols = X.select_dtypes(exclude="number").columns.tolist()

preprocessor = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc", StandardScaler())]), num_cols),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("oh", OneHotEncoder(handle_unknown="ignore"))]), cat_cols),
])
print("Numerical features :", len(num_cols))
print("Categorical features:", len(cat_cols), cat_cols)
```

```
Numerical features : 10
Categorical features: 5 ['city', 'locality_tier', 'property_type', 'furnishing', 'facing']
```

Figure 2.7 — Data preprocessing pipeline (code and output)

2.6 Train-Test Split

The dataset was split into training and testing sets using an 80:20 ratio and random_state=42.

Subset	Records	Purpose
Training set	1,920	Fit model parameters
Testing set	480	Evaluate on unseen records
Total	2,400	Complete dataset

The test set was held out from model fitting and used for final performance comparison. A fixed random state makes the split reproducible.

03_model_training.ipynb ×

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Feature matrix shape:", X.shape)
print("Training records:", len(X_train))
print("Testing records :", len(X_test))
```

```
Feature matrix shape: (2400, 15)
Training records: 1920
Testing records : 480
```

Figure 2.8 — Train-test split (code and output)

2.7 ANN Architecture and Training Configuration

The ANN was implemented using Scikit-learn's MLPRegressor. It uses two hidden layers containing 128 and 64 neurons, with ReLU activation and the Adam optimiser.

Parameter	Configuration
Estimator	MLPRegressor
Hidden layers	(128, 64)
Activation	ReLU
Solver	Adam
Regularisation (alpha)	0.001
Initial learning rate	0.001
Maximum iterations	500
Early stopping	Enabled
Validation fraction	0.10
No-improvement patience	20 iterations
Random state	42

Early stopping monitors performance on a validation subset and stops training when improvement stalls. Regularisation helps limit model complexity. For the original-target ANN, the price was expressed in ₹ lakh (price ÷ 100,000) during training and converted back to rupees for evaluation. This is a change of units only, not a non-linear transformation; a first trial that used raw rupee values failed to train because the target values were extremely large relative to the network's initial output scale.

03_model_training.ipynb ×

```
from sklearn.neural_network import MLPRegressor

def build_ann():
    return MLPRegressor(hidden_layer_sizes=(128, 64), activation="relu",
                        solver="adam", alpha=0.001, learning_rate_init=0.001,
                        max_iter=500, early_stopping=True,
                        validation_fraction=0.1, n_iter_no_change=20,
                        random_state=42)

# price is expressed in INR lakh while training (a change of units only)
ann = Pipeline([("prep", preprocessor), ("model", build_ann())])
ann.fit(X_train, y_train / 1e5)
print("ANN training stopped after", ann.named_steps["model"].n_iter_, "iterations")
```

```
ANN training stopped after 265 iterations
```

Figure 2.9 — ANN configuration and training (code and output)

2.7.1 Log Transformation of the Target

Because prices vary widely and are right-skewed, a second ANN experiment used y_log = log(1 + y). Predictions were returned to the original price scale using y_pred = exp(y_log_pred) − 1. This experiment tested whether learning the transformed target improved prediction performance.

2.8 Comparison Models

Linear Regression was used as a baseline. Random Forest Regression was also evaluated because it can learn non-linear relationships and feature interactions through an ensemble of decision trees. The Random Forest configuration used 300 trees, min_samples_leaf=2, random_state=42, and n_jobs=-1.

Four variants were compared: Linear Regression, original-target ANN, log-target ANN, and Random Forest. Each was evaluated on the same test set.

04_model_evaluation.ipynb ×

```python
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

ann_log = Pipeline([("prep", preprocessor), ("model", build_ann())])
ann_log.fit(X_train, np.log1p(y_train))
lr = Pipeline([("prep", preprocessor), ("model", LinearRegression())]).fit(X_train, y_train)
rf = Pipeline([("prep", preprocessor), ("model", RandomForestRegressor(
        n_estimators=300, min_samples_leaf=2, random_state=42, n_jobs=-1))]).fit(X_train, y_train)

preds = {
    "Linear Regression":   lr.predict(X_test),
    "Original-target ANN": ann.predict(X_test) * 1e5,
    "Log-target ANN":      np.expm1(ann_log.predict(X_test)),
    "Random Forest":       rf.predict(X_test),
}
results = pd.DataFrame([{
    "Model": k, "MAE": mean_absolute_error(y_test, p),
    "RMSE": mean_squared_error(y_test, p) ** 0.5, "R2": r2_score(y_test, p)}
    for k, p in preds.items()])
pd.options.display.float_format = "{:,.2f}".format
print(results.to_string(index=False))
```

```
               Model          MAE         RMSE   R2
   Linear Regression 4,488,497.82 8,542,332.93 0.73
Original-target ANN 2,398,150.15 4,688,552.17 0.92
    Log-target ANN 2,827,231.87 5,790,054.49 0.87
     Random Forest 2,588,345.94 5,842,015.23 0.87
```

Figure 2.10 — Model comparison (code and output)

2.9 Evaluation Metrics

2.9.1 Mean Absolute Error (MAE)

MAE measures the average absolute difference between actual and predicted prices. A lower value indicates smaller average errors.

$$MAE = (1/n) \Sigma |y_i - \hat{y}_i|$$

2.9.2 Root Mean Squared Error (RMSE)

RMSE is the square root of the mean squared prediction error. Because errors are squared before averaging, large errors have a stronger effect.

$$RMSE = \sqrt{[(1/n) \Sigma (y_i - \hat{y}_i)^2]}$$

2.9.3 Coefficient of Determination (R^2)

R^2 compares model squared prediction errors with variation around the mean target value. A value closer to 1 generally indicates better explanatory performance; a negative value indicates performance worse than the mean-prediction baseline.

$$R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$$

CHAPTER 3

IMPLEMENTATION

3.1 Tools and Technologies

Technology	Purpose
Python	Main programming language
Pandas and NumPy	Data manipulation and numerical operations
Matplotlib	Visualisation of data and model performance
Scikit-learn	Preprocessing, model training, and evaluation
Joblib	Saving and loading the trained pipeline
Microsoft Word	Project report preparation

3.2 Project Workflow

1. Generate and store the housing dataset.
2. Explore the data and identify missing values and price-distribution characteristics.
3. Clean and preprocess the dataset.
4. Build numerical and categorical preprocessing pipelines.
5. Train Linear Regression, two ANN variants, and Random Forest.
6. Evaluate the models using MAE, RMSE, and R^2.
7. Select the final model based on test-set performance.
8. Save the selected pipeline for reuse in predictions.



Figure 3.1 — Project Workflow Directory

3.3 Model Training and Saving

The models were trained on the training subset and evaluated on the held-out test subset. Preprocessing transformations were combined with each model in a single pipeline so the same feature preparation could be applied at prediction time.

The selected ANN pipeline was saved as ann_pipeline.joblib. Saving the trained pipeline avoids retraining every time a prediction is needed and preserves the preprocessing configuration used during model development.

3.4 Project Components

File / component	Responsibility
housing_data.csv	Generated housing dataset
run_models.py	Data preparation, model training, evaluation and figure generation
results.json	Saved metrics and summary statistics
figs/	Figures used in the report
ann_pipeline.joblib	Saved ANN preprocessing and model pipeline

A web interface for entering property details has not been built in this project. It is listed as future work in Chapter 5.

3.5 User Input and Prediction Process

1. The property details are organised into the feature structure expected by the model.
2. The saved pipeline is loaded.
3. The pipeline applies the required preprocessing transformations.
4. The model generates an estimated property price.
5. The estimated price is returned to the user.

The prediction is an estimate based on patterns in the training data, not a guaranteed selling price. Market demand, property condition, legal status, and other unobserved factors may influence actual value.

3.6 Sample Prediction

To check that the saved pipeline accepts new property details and returns a price, a sample property was passed to it.

Input	Value
City	Bengaluru
Locality tier	Mid
Property type	Apartment
Area (sq ft)	1250

Input	Value
Bedrooms / Bathrooms	3 / 2
Balconies	2
Age (years)	5
Floor / Total floors	7 / 15
Furnishing	Semi-Furnished
Parking	1
Facing	East
Near metro / Lift	Yes / Yes
Estimated price (ANN)	₹1,13,77,000

The estimated price for this sample is approximately ₹113.8 lakh. Producing a prediction confirms that the pipeline works end to end, but it does not itself establish model accuracy; accuracy is assessed on the held-out test set in Chapter 4.

The code used to load the saved pipeline and make this prediction is shown in Figure 3.2.

predict.py

```python
import joblib

model = joblib.load("ann_pipeline.joblib")

sample = pd.DataFrame([{
    "city": "Bengaluru", "locality_tier": "Mid", "property_type": "Apartment",
    "area_sqft": 1250, "bedrooms": 3, "bathrooms": 2, "balconies": 2,
    "age_years": 5, "floor": 7, "total_floors": 15, "furnishing": "Semi-Furnished",
    "parking": 1, "facing": "East", "near_metro": 1, "lift": 1,
}])
price = model.predict(sample)[0] * 1e5          # back to INR
print(f"Estimated price: Rs {price:,.0f}  ({price / 1e5:,.1f} lakh)")
```

```
Estimated price: Rs 11,377,323  (113.8 lakh)
```

Figure 3.2 — Sample prediction (code and output)

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Overview of Model Evaluation

Four model variants were evaluated on the same test set of 480 records. The comparison used MAE, RMSE, and R^2 to assess average absolute error, the impact of large errors, and explanatory performance.

4.2 Comparison of Model Performance

Model	MAE (₹)	RMSE (₹)	R^2
Linear Regression	44,88,497.82	85,42,332.93	0.7273
Original-target ANN	23,98,150.15	46,88,552.17	0.9179
Log-target ANN	28,27,231.87	57,90,054.49	0.8747
Random Forest	25,88,345.94	58,42,015.23	0.8725

The original-target ANN achieved the lowest MAE (approximately ₹23.98 lakh), the lowest RMSE and the highest R^2 (0.9179) of the four models. Random Forest and the log-target ANN performed similarly to each other (R^2 of 0.8725 and 0.8747), and all three non-linear models clearly outperformed the Linear Regression baseline (R^2 0.7273), whose MAE was about 1.9 times that of the best ANN.

These results show why models should be compared using multiple metrics. The MAE describes the average size of an error, while RMSE places greater weight on large errors, which matter in the high-priced segment of this dataset.

4.3 Analysis of ANN Performance

The original-target ANN achieved an MAE of ₹23,98,150.15 and R^2 of 0.9179. After transforming the target using log(1 + price), the log-target ANN achieved an MAE of ₹28,27,231.87 and R^2 of 0.8747. The log-target MAE was approximately 17.9% higher than that of the original-target ANN.

Unlike many skewed-target problems, the log transformation did not improve results here. A likely reason is that the metrics are measured in rupees: a log-trained model minimises relative error, so it is relatively accurate for cheap and expensive homes alike but pays less attention to the large rupee errors on expensive properties, which dominate MAE and RMSE. The original-target ANN was trained with the squared error on the rupee scale and therefore concentrated on those expensive properties. The result is specific to this dataset and should be re-checked on real data. Training stopped early after 265 iterations for the original-target ANN and 293 iterations for the log-target ANN.

4.4 Actual-versus-Predicted Analysis

Actual-versus-predicted plots help show how closely model outputs align with observed prices. Points closer to the diagonal reference line indicate predictions closer to actual values; points farther from the line represent larger errors.

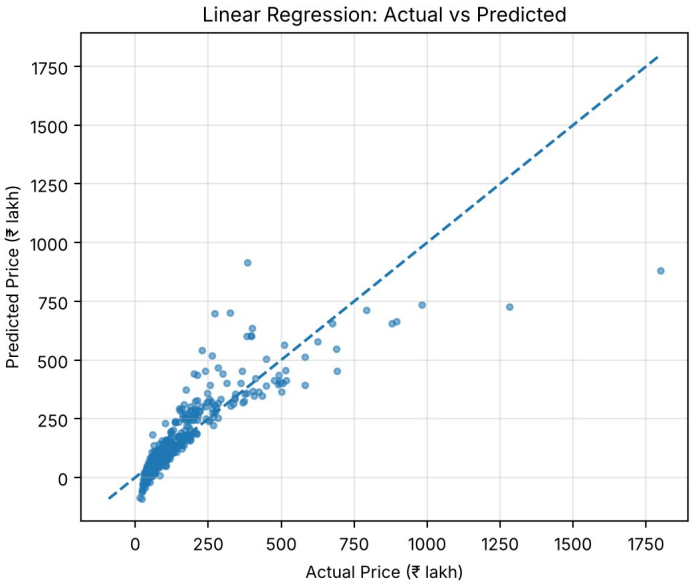


Figure 4.1 — Linear Regression: actual vs predicted

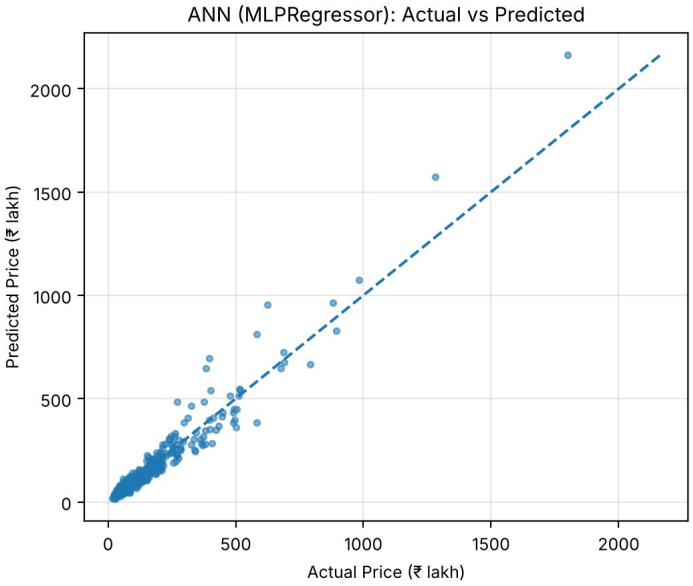


Figure 4.2 — Original-target ANN: actual vs predicted

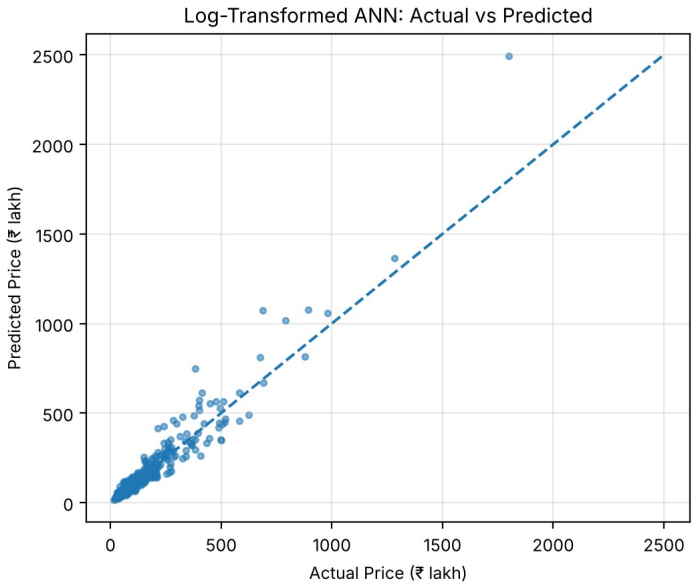


Figure 4.3 — Log-target ANN: actual vs predicted

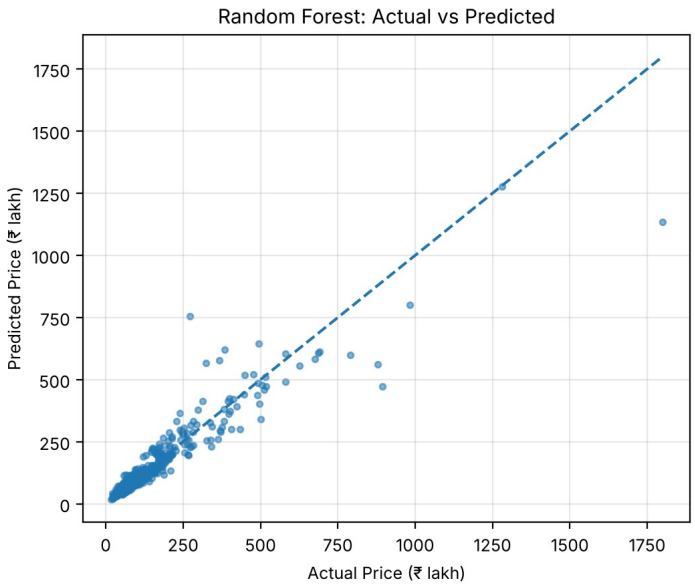


Figure 4.4 — Random Forest: actual vs predicted

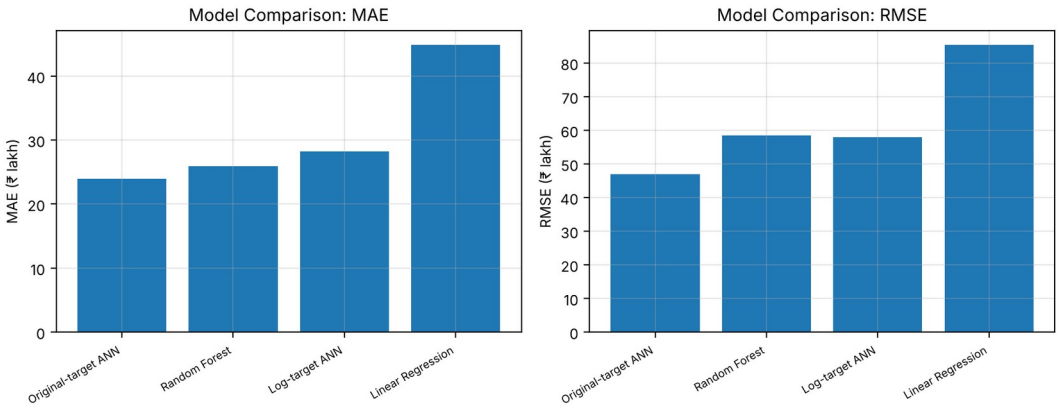


Figure 4.5 — Model error comparison (MAE and RMSE)

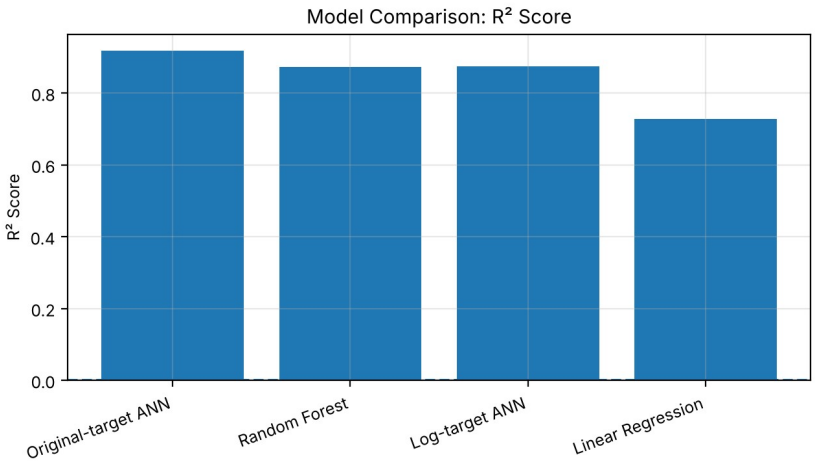


Figure 4.6 — R² comparison

4.5 Price-Segment Performance

To examine performance across price ranges, the test set was divided into budget, mid-range, and premium segments.

Price segment	Test records	Original-target ANN MAE	Log-target ANN MAE	Random Forest MAE
Budget (≤ ₹50 lakh)	73	₹9,08,221	₹8,64,787	₹8,25,684
Mid-range (> ₹50 lakh–1.5 crore)	266	₹12,83,548	₹14,90,962	₹13,20,359
Premium (> ₹1.5 crore)	141	₹52,72,256	₹63,64,156	₹58,93,019

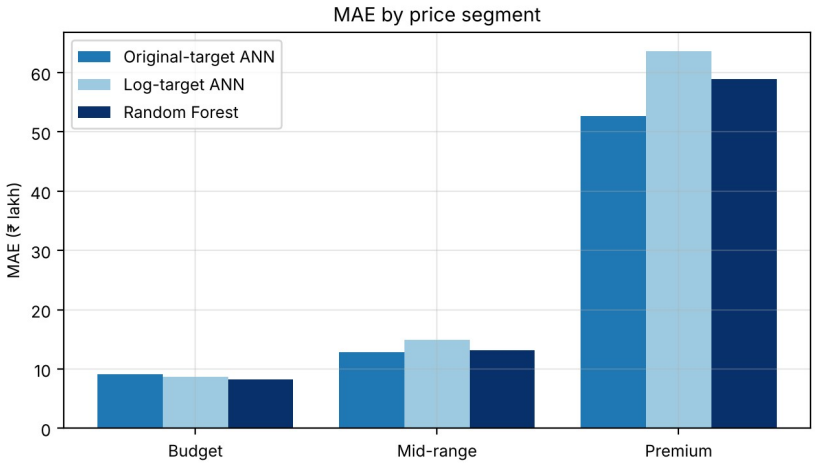


Figure 4.7 — MAE by price segment

Random Forest had the lowest MAE in the budget segment, while the original-target ANN had the lowest MAE in the mid-range and premium segments. Errors were smallest in the budget segment and largest in the premium segment, where the MAE of the best model was ₹52.7 lakh. This indicates that prediction difficulty increases for high-value properties and that overall metrics should be interpreted alongside segment-level results.

4.6 Feature Importance

Random Forest feature importances give an indication of which attributes the models rely on most. Importances for one-hot encoded columns were summed back to their original feature.

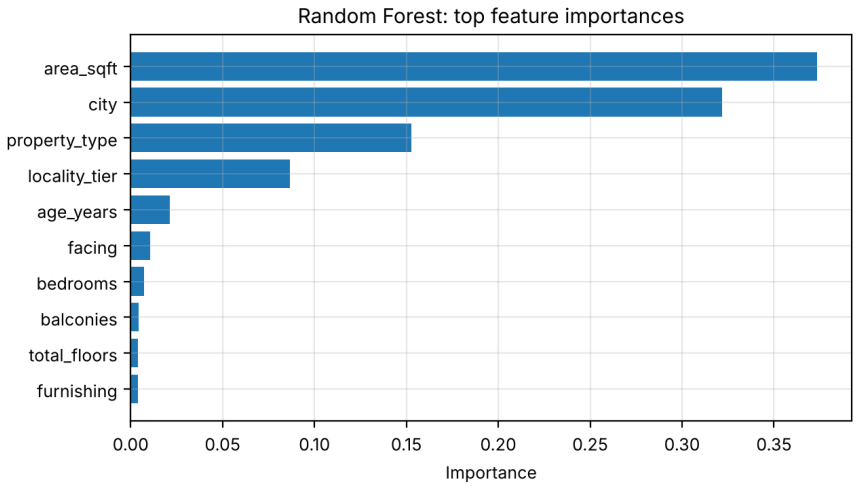


Figure 4.8 — Random Forest: top feature importances

Area (37.4%), city (32.2%) and property type (15.3%) together account for most of the importance, followed by locality tier (8.7%). This is consistent with how the simulated prices were generated and with the common view that size and location are the main drivers of housing prices. Because the data are simulated, these importances describe the dataset and are not evidence about the real market.

4.7 Challenges in Predicting High-Value Properties

The right-skewed price distribution means there are few very expensive properties, particularly large villas in the most expensive cities. Sparse representation limits the model's ability to learn reliable pricing patterns for them.

The largest single error of the original-target ANN occurred for a Mumbai villa with an actual price of approximately ₹17.99 crore, which was predicted at approximately ₹21.64 crore. For the same property, Random Forest predicted approximately ₹11.35 crore. This illustrates that substantial errors remain for rare high-value properties. Possible contributing factors include limited examples, extreme prices, and information about condition, view or demand that is not captured by the dataset.

4.8 Selection of the Final Model

The original-target ANN was selected as the final model because it achieved the lowest MAE, the lowest RMSE and the highest R^2 among the four evaluated models, and the lowest MAE in the mid-range and premium segments. Random Forest remained slightly better for budget

properties. Compared with Random Forest, the ANN's MAE was 7.3% lower and its RMSE was 19.7% lower. The selected pipeline was saved as ann_pipeline.joblib.

4.9 Summary of Results

The results show that machine-learning models can learn useful relationships between property features and prices, but performance differs by algorithm and price segment. The ANN and Random Forest clearly outperformed Linear Regression, and the ANN achieved the best overall scores on this dataset. Rare high-value properties remain challenging, motivating larger and more representative datasets and additional property information.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project applied machine-learning techniques to estimate housing prices from structured property information. The work included dataset generation and exploration, preprocessing, ANN implementation, comparison with Linear Regression and Random Forest, and evaluation on a held-out test set.

The original-target ANN achieved the lowest MAE (₹23,98,150.15), the lowest RMSE and the highest R^2 (0.9179) and was selected as the final model. The log-target ANN and Random Forest performed comparably to each other, and Linear Regression performed worst, highlighting the importance of non-linear models for this problem and the value of considering multiple evaluation metrics.

The project provided practical experience in data preparation, neural-network regression, model evaluation and pipeline design. It also identified limitations associated with sparse high-value examples and the use of a simulated dataset.

5.2 Limitations

- The dataset is simulated and contains 2,400 records; findings should be validated on real housing listings before being used for any valuation.
- The price distribution is skewed, and very high-value properties are underrepresented.
- Structured attributes do not fully capture construction quality, view, legal status, maintenance, or hidden defects.
- Local demand, interest rates and changing market conditions may affect actual selling prices.
- The prediction is an estimate and should not be treated as a guaranteed market value.
- The relative ranking of the log-target ANN and Random Forest differs by metric and segment, and results come from a single train-test split.

5.3 Future Scope

5.3.1 Use of Real Housing Data

The most important next step is to repeat the experiments on a real housing-listings dataset, including more cities, localities and a fuller range of property types, and to compare the results with those reported here.

5.3.2 Web Application

A Streamlit or similar web application could let users enter property details and view an estimated price using the saved pipeline, together with a clear note that the figure is an estimate.

5.3.3 Image and Location Features

Property photographs could be analysed with a convolutional neural network to describe visible condition, and geographic information such as distance to schools, hospitals and transport could be added. A multimodal model could combine these with the structured attributes, but this would need a dataset with paired images, locations and reliable price labels.

5.3.4 Model Optimisation

Future experiments could investigate alternative ANN architectures, systematic hyperparameter tuning, cross-validation, and additional regression algorithms such as gradient boosting. Segment-wise evaluation and uncertainty estimates could also help communicate when predictions are less reliable.

5.4 Final Remarks

The project demonstrates a practical workflow for housing price prediction and illustrates the importance of comparing models using appropriate evaluation metrics. ANN experiments formed the central analytical component and gave the best overall performance on the dataset used. Applying the same workflow to real market data is a promising direction for further work.

REFERENCES

1. Scikit-learn Developers. MLPRegressor — Multi-layer Perceptron Regressor. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html
2. Scikit-learn Developers. RandomForestRegressor. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html
3. Scikit-learn Developers. LinearRegression. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
4. Scikit-learn Developers. ColumnTransformer. https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html
5. Scikit-learn Developers. Model evaluation: regression metrics. https://scikit-learn.org/stable/modules/model_evaluation.html
6. The pandas Development Team. pandas Documentation. https://pandas.pydata.org/docs/
7. Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/
8. Joblib Developers. Joblib: running Python functions as pipeline jobs. https://joblib.readthedocs.io/